

A Practical Guide to Neural architecture search

TOPIC -- NEURAL ARCHITECTURE SEARCH

-- 01 --

The search space defines the type(s) of ANN that can be designed and optimized. The search strategy defines the approach used to explore the search space. The performance estimation strategy evaluates the performance of a possible ANN from its design (without constructing and training it). NAS is closely related to hyperparameter optimization and meta-learning and is a subfield of automated machine learning (AutoML).

-- 02 --

Evolution An alternative approach to NAS is based on evolutionary algorithms, which has been employed by several groups. An Evolutionary Algorithm for Neural Architecture Search generally performs the following procedure. First a pool consisting of different candidate architectures along with their validation scores (fitness) is initialised. At each step the architectures in the candidate pool are mutated (e.g.: 3x3 convolution instead of a 5x5 convolution). Next the new architectures are trained from scratch for a few epochs and their validation scores are obtained. This is followed by replacing the lowest scoring architectures in the candidate pool with the better, newer architectures. This procedure is repeated multiple times and thus the candidate pool is refined over time. Mutations in the context of evolving ANNs are operations such as adding or removing a layer, which include changing the type of a layer (e.g., from convolution to pooling), changing the hyperparameters of a layer, or changing the training hyperparameters. On CIFAR-10 and ImageNet, evolution and RL performed comparably, while both slightly outperformed random search.

-- 03 --

Neural architecture search benchmarks Neural architecture search often requires large computational resources, due to its expensive training and evaluation phases. This further leads to a large carbon footprint required for the evaluation of these methods. To overcome this limitation, NAS benchmarks have been introduced, from which one can either query or predict the final performance of neural architectures in seconds. A NAS benchmark is defined as a dataset with a fixed train-test split, a search space, and a fixed training pipeline (hyperparameters). There are primarily two types of NAS benchmarks: a surrogate NAS benchmark and a tabular NAS benchmark. A surrogate

benchmark uses a surrogate model (e.g.: a neural network) to predict the performance of an architecture from the search space. On the other hand, a tabular benchmark queries the actual performance of an architecture trained up to convergence. Both of these benchmarks are queryable and can be used to efficiently simulate many NAS algorithms using only a CPU to query the benchmark instead of training an architecture from scratch.

-- 04 --

Neural architecture search (NAS) is a technique for automating the design of artificial neural networks (ANN), a widely used model in the field of machine learning. NAS has been used to design networks that are on par with or outperform hand-designed architectures. Methods for NAS can be categorized according to the search space, search strategy and performance estimation strategy used:

-- 05 --

Reinforcement learning Reinforcement learning (RL) can underpin a NAS search strategy. Barret Zoph and Quoc Viet Le applied NAS with RL targeting the CIFAR-10 dataset and achieved a network architecture that rivals the best manually-designed architecture for accuracy, with an error rate of 3.65, 0.09 percent better and 1.05x faster than a related hand-designed model. On the Penn Treebank dataset, that model composed a recurrent cell that outperforms LSTM, reaching a test set perplexity of 62.4, or 3.6 perplexity better than the prior leading system. On the PTB character language modeling task it achieved bits per character of 1.214. Learning a model architecture directly on a large dataset can be a lengthy process. NASNet addressed this issue by transferring a building block designed for a small dataset to a larger dataset. The design was constrained to use two types of convolutional cells to return feature maps that serve two main functions when convoluting an input feature map: normal cells that return maps of the same extent (height and width) and reduction cells in which the returned feature map height and width is reduced by a factor of two. For the reduction cell, the initial operation applied to the cell's inputs uses a stride of two (to reduce the height and width). The learned aspect of the design included elements such as which lower layer(s) each higher layer took as input, the transformations applied at that layer and to merge multiple outputs at each layer. In the studied example, the best convolutional layer (or "cell") was designed for the CIFAR-10 dataset and then applied to the ImageNet dataset by stacking copies of this cell, each with its own parameters. The approach yielded accuracy of 82.7% top-1 and 96.2% top-5. This exceeded the best human-invented architectures at a cost of 9 billion fewer FLOPS—a reduction of

28%. The system continued to exceed the manually-designed alternative at varying computation levels. The image features learned from image classification can be transferred to other computer vision problems. E.g., for object detection, the learned cells integrated with the Faster-RCNN framework improved performance by 4.0% on the COCO dataset. In the so-called Efficient Neural Architecture Search (ENAS), a controller discovers architectures by learning to search for an optimal subgraph within a large graph. The controller is trained with policy gradient to select a subgraph that maximizes the validation set's expected reward. The model corresponding to the subgraph is trained to minimize a canonical cross entropy loss. Multiple child models share parameters, ENAS requires fewer GPU-hours than other approaches and 1000-fold less than "standard" NAS. On CIFAR-10, the ENAS design achieved a test error of 2.89%, comparable to NASNet. On Penn Treebank, the ENAS design reached test perplexity of 55.8.

-- 06 --

Wistuba, Martin; Rawat, Ambrish; Pedapati, Tejaswini (2019-05-04). "A Survey on Neural Architecture Search". arXiv:1905.01392 [cs.LG]. Elsken, Thomas; Metzen, Jan Hendrik; Hutter, Frank (August 8, 2019). "Neural Architecture Search: A Survey". *Journal of Machine Learning Research*. 20 (55): 1–21. arXiv:1808.05377. Liu, Yuqiao; Sun, Yanan; Xue, Bing; Zhang, Mengjie; Yen, Gary G; Tan, Kay Chen (2021). "A Survey on Evolutionary Neural Architecture Search". *IEEE Transactions on Neural Networks and Learning Systems*. 34 (2): 1–21. arXiv:2008.10937. doi:10.1109/TNNLS.2021.3100554. PMID 34357870. S2CID 221293236. White, Colin; Safari, Mahmoud; Sukthanker, Rhea; Ru, Binxin; Elsken, Thomas; Zela, Arber; Dey, Debadeepta; Hutter, Frank (2023-01-25). "Neural Architecture Search: Insights from 1000 Papers". arXiv:2301.08727 [cs.LG].